Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Конструирование программного обеспечения»

Тема: «Создание ключевых функций»

Выполнила:

Студентка 2 курса

250851

Группы ПО-17

Бычик К.В.

Проверил:

Кулик А.Д.

Брест 2026

Цель работы: написать основную логику программы, реализовав игровой процесс через классы и используя настройки из предыдущих лабораторных работ.

Ход работы

Вариант 2

2. Игра представляет собой последовательное посещение предметов и самостоятельную работу дома. Реализовать классы Student, Subject и Action (занятие студента, отдых, учёба, решение задач и т.д.). Игрок должен выбирать, какие предметы посещать, какие прогуливать и чем заниматься дома, увеличивая свои характеристики. Через определённое количество дней должна начинаться сессия, где оценка зависит от набранных игроком характеристик, а также он может ответить на вопросы по предмету, чтобы повысить получаемый балл.

Код программы:

```
#define NOMINMAX
#include <iostream>
#include <string>
#include <fstream>
#include <list>
#include <set>
#include <vector>
#include <algorithm>
#include <windows.h>
using namespace std;

const string VERSION = "3.0";
const string SETTINGS_FILE = "settings.txt";
const string STUDENTS_FILE = "students.txt";
const string SPECIALITY_FILE = "specialities.txt";

struct Settings {
    string speciality = "Программная инженерия";
    string studentName = "Кристина";
    int subjectsCount = 5;
};

void clearInput() { cin.clear(); cin.ignore(10000, '\n'); }
void waitEnter() { cout << "\nНажмите Enter..."; cin.get(); }

void ensureFile(const string& name, const string& content) {
    ifstream f(name);
    if (!f.good()) ofstream(name) << content;
}

void initFiles(const Settings& d) {
    ensureFile(SETTINGS_FILE,
        "speciality=" + d.speciality + "\nstudentName=" + d.studentName +
        "\nsubjectsCount=" + to_string(d.subjectsCount) + "\n");
    ensureFile(STUDENTS_FILE, "Кристина\nИван\nАнна\nПётр\nОльга\n");
    ensureFile(SPECIALITY_FILE,
        "Программная инженерия\nИнформационные технологии\n"
        "Искусственный интеллект\nКибербезопасность\nВеб-разработка\n");
}

void loadSettings(Settings& s) {
    ifstream in(SETTINGS_FILE);
    string line;
    while (getline(in, line)) {
        size_t p = line.find('=');
        if (p == string::npos) continue;
        string k = line.substr(0, p), v = line.substr(p + 1);
        if (k == "speciality")       s.speciality = v;
        else if (k == "studentName") s.studentName = v;
        else if (k == "subjectsCount")
            try { s.subjectsCount = stoi(v); }
        catch (...) {}
    }
}

void saveSettings(const Settings& s) {
    ofstream out(SETTINGS_FILE);
    out << "speciality=" << s.speciality << "\n"
        << "studentName=" << s.studentName << "\n"
        << "subjectsCount=" << s.subjectsCount << "\n";
```

```
}

template <typename C>
void loadLines(const string& file, C& c) {
    c.clear();
    ifstream in(file);
    string line;
    while (getline(in, line)) if (!line.empty()) c.insert(c.end(), line);
}

template <typename C>
bool chooseFrom(const C& c, string& out) {
    if (c.empty()) return false;
    int i = 1;
    for (const auto& v : c) cout << "  " << i++ << ". " << v << "\n";
    cout << "Выберите номер (0 - отмена): ";
    int n; cin >> n;
    if (cin.fail() || n < 0 || n >(int)c.size()) { clearInput(); return false; }
    if (n == 0) return false;
    auto it = c.begin(); advance(it, n - 1); out = *it;
    return true;
}

// ================= КЛАССЫ ИГРЫ =================

class Subject {
    string name;
    int progress = 0;
public:
    Subject(string n = "") : name(n) {}
    string getName() const { return name; }
    int getProgress() const { return progress; }
    void study(int a) { progress = min(100, progress + a); }
};

class Action {
public:
    enum Type { ATTEND, SKIP, STUDY_HOME, REST };
    Type type;
    string name;
    int k, e, m;
    Action(Type t, string n, int k_, int e_, int m_)
        : type(t), name(n), k(k_), e(e_), m(m_) {
    }
};

class Student {
    string name, speciality;
    int knowledge = 30, energy = 80, motivation = 60;
    vector<Subject> subjects;
public:
    Student(string n, string s) : name(n), speciality(s) {}
    void addSubject(const Subject& s) { subjects.push_back(s); }
    vector<Subject>& getSubjects() { return subjects; }
    string getName() const { return name; }
    string getSpeciality() const { return speciality; }
    int getKnowledge() const { return knowledge; }

    void applyAction(const Action& a, Subject* subj = nullptr) {
        knowledge = max(0, min(100, knowledge + a.k));
        energy = max(0, min(100, energy + a.e));
        motivation = max(0, min(100, motivation + a.m));
        if (subj) {
            if (a.type == Action::ATTEND)          subj->study(15);
            else if (a.type == Action::STUDY_HOME) subj->study(10);
        }
    }

    void showStatus() const {
        cout << "\n--- " << name << " (" << speciality << ") ---\n"
            << "Знания: " << knowledge << "  Энергия: " << energy
            << "  Мотивация: " << motivation << "\n";
        for (size_t i = 0; i < subjects.size(); ++i)
            cout << "  " << i + 1 << ". " << subjects[i].getName()
            << " — " << subjects[i].getProgress() << "%\n";
    }
};

class Game {
    Student student;
    int totalDays = 10, currentDay = 1;
public:
    Game(const Settings& s) : student(s.studentName, s.speciality) {
        static const char* names[] = {
```

```
            "Математика", "Программирование", "Физика", "Английский", "История",
            "Базы данных", "Алгоритмы", "Философия", "Экономика", "Право"
        };
        int count = max(1, min(10, s.subjectsCount));
        for (int i = 0; i < count; ++i)
            student.addSubject(Subject(names[i % 10]));
    }

    void run() {
        Action attend(Action::ATTEND, "Посетить", +10, -15, +5);
        Action skip(Action::SKIP, "Прогулять", -5, +10, -10);
        Action studyHome(Action::STUDY_HOME, "Учиться", +7, -10, +2);
        Action rest(Action::REST, "Отдохнуть", 0, +20, +10);

        while (currentDay <= totalDays) {
            student.showStatus();
            cout << "\n--- День " << currentDay << "/" << totalDays << " ---\n"
                << "1. Посетить  2. Прогулять  3. Учиться  4. Отдохнуть\nВыбор: ";
            int c; cin >> c;
            if (cin.fail()) { clearInput(); continue; }

            Subject* target = nullptr;
            auto& subjects = student.getSubjects();
            if (c >= 1 && c <= 3) {
                cout << "Выберите предмет (0 - отмена):\n";
                for (size_t i = 0; i < subjects.size(); ++i)
                    cout << "  " << i + 1 << ". " << subjects[i].getName() << "\n";
                int idx; cin >> idx;
                if (cin.fail() || idx < 0 || idx >(int)subjects.size()) {
                    clearInput(); continue;
                }
                if (idx == 0) continue;
                target = &subjects[idx - 1];
            }

            if (c == 1) student.applyAction(attend, target);
            else if (c == 2) student.applyAction(skip, target);
            else if (c == 3) student.applyAction(studyHome, target);
            else if (c == 4) student.applyAction(rest);
            else continue;

            currentDay++;
        }
        session();
    }

private:
    void session() {
        cout << "\n========================================\n"
            << "           СЕССИЯ НАЧАЛАСЬ!\n"
            << "========================================\n";

        int total = 0;
        auto& subjects = student.getSubjects();

        for (auto& subj : subjects) {
            int base = max(1, min(5, (subj.getProgress() + student.getKnowledge()) / 40));

            cout << "\n=== " << subj.getName()
                << " (прогресс " << subj.getProgress() << "%) ===\n";

            int correct = 0, a;

            cout << "1. Вы изучили больше половины материала? (1/0): ";
            cin >> a;
            if (a == 1 && subj.getProgress() >= 50) correct++;

            cout << "2. Вы посещали большинство занятий? (1/0): ";
            cin >> a;
            if (a == 1 && subj.getProgress() >= 60) correct++;

            cout << "3. Вы готовы к экзамену? (1/0): ";
            cin >> a;
            if (a == 1 && subj.getProgress() >= 70) correct++;

            int bonus = min(2, correct);
            int grade = min(5, base + bonus);

            cout << "Оценка: " << grade
                << " (база " << base << " + бонус " << bonus << ")\n";
            total += grade;
        }

        double avg = subjects.empty() ? 0.0 : (double)total / subjects.size();
```

```
        cout << "\nСредний балл: " << avg << "\n";
        if (avg >= 4.5)      cout << "Отличник! Стипендия обеспечена.\n";
        else if (avg >= 3.5) cout << "Хороший результат.\n";
        else if (avg >= 2.5) cout << "Сессия сдана, но есть долги.\n";
        else                 cout << "Вас отчислили...\n";
    }
};

// ================ МЕНЮ ================

void startGame(const Settings& s) { Game g(s); g.run(); waitEnter(); }
void loadGame() { cout << "\nЗагрузка недоступна.\n"; waitEnter(); }
void aboutProgram() {
    cout << "\n=== О программе ===\nВерсия: " << VERSION
        << "\nСтудент: Бычик К.В., группа ПО-17\n";
    waitEnter();
}

void settingsMenu(Settings& s, const list<string>& students, const set<string>& specialities) {
    int choice = 0;
    while (choice != 4) {
        cout << "\n--- Настройки ---\n"
            << "1. Специальность (" << s.speciality << ")\n"
            << "2. Имя студента (" << s.studentName << ")\n"
            << "3. Предметов (" << s.subjectsCount << ")\n"
            << "4. Назад\nВыбор: ";
        cin >> choice;
        if (cin.fail()) { clearInput(); continue; }

        if (choice == 1) {
            clearInput();
            if (chooseFrom(specialities, s.speciality)) saveSettings(s);
        }
        else if (choice == 2) {
            clearInput();
            if (chooseFrom(students, s.studentName)) saveSettings(s);
        }
        else if (choice == 3) {
            int n; cin >> n;
            if (cin.fail() || n <= 0) { clearInput(); cout << "Ошибка!\n"; }
            else { s.subjectsCount = n; saveSettings(s); }
        }
    }
}

int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    Settings settings;
    initFiles(settings);
    loadSettings(settings);

    list<string> students;
    set<string> specialities;
    loadLines(STUDENTS_FILE, students);
    loadLines(SPECIALITY_FILE, specialities);

    int choice = 0;
    while (choice != 5) {
        cout << "\n*** Моя жизнь в университете ***\n"
            << "1. Начать игру\n2. Загрузить игру\n3. Настройки\n"
            << "4. О программе\n5. Выход\nВыбор: ";
        cin >> choice;
        if (cin.fail()) { clearInput(); continue; }

        switch (choice) {
        case 1: startGame(settings); break;
        case 2: loadGame(); break;
        case 3: settingsMenu(settings, students, specialities); break;
        case 4: aboutProgram(); break;
        case 5: cout << "Выход...\n"; break;
        default: cout << "Нет такого пункта.\n";
        }
    }
    return 0;
}
```

Вывод: в ходе лабораторной работы была реализована игровая логика с использованием классов, что позволило связать сохраненные настройки с игровым процессом и добавить функционал при выборе пункта «Начать игру».

Контрольные вопросы:

0. Алгоритм работы программы

1. Инициализация консоли (кодировка 1251, локаль).
   Создание файлов настроек/студентов/специальностей, если их нет (initFiles).

2. Загрузка настроек из settings.txt (loadSettings).
3. Загрузка списков студентов и специальностей (loadLines).
4. Главное меню (цикл): Начать игру / Загрузить / Настройки / О программе / Выход.
5. При старте игры создаётся Student с предметами (Subject), запускается Game::run().
6. Игровой цикл по дням (10 дней): показ статуса, выбор действия (Посетить/Прогулять/Учиться/Отдохнуть), применение Action к студенту и предмету, изменение характеристик (знания, энергия, мотивация, прогресс).
7. После последнего дня - session(): по каждому предмету считается база оценки из прогресса и знаний, начисляется бонус за ответы, выводится средний балл и вердикт.
8. Возврат в меню.

1. Классы, объекты, поля, методы

- **Классы:** Subject, Action, Student, Game, а также структура Settings.
- **Объекты:** student (в Game), subjects (вектор Subject), attend/skip/studyHome/rest (объекты Action), settings, g (объект Game).
- **Поля:**
  - Subject: name, progress;
  - Action: type, name, k, e, m;
  - Student: name, speciality, knowledge, energy, motivation, subjects;
  - Game: student, totalDays, currentDay.
- **Методы:**
  - Subject: getName, getProgress, study;
  - Student: addSubject, getSubjects, getName, getSpeciality, getKnowledge, applyAction, showStatus;
  - Game: run, session (private);
  - вспомогательные функции: clearInput, waitEnter, ensureFile, initFiles, loadSettings, saveSettings, loadLines, chooseFrom, startGame, loadGame, aboutProgram, settingsMenu, main.

2. Основные принципы ООП

- **Инкапсуляция** — поля классов закрыты (private по умолчанию), доступ через методы (getName, getProgress, applyAction).
- **Абстракция** — классы моделируют реальные сущности (студент, предмет, действие), скрывая детали реализации.
- **Наследование** — в явном виде не используется (можно было бы выделить базовый класс для действий/сущностей).
- **Полиморфизм** — не реализован через виртуальные функции; частично проявляется через шаблоны (loadLines, chooseFrom) и перегрузку/переопределение поведения через параметры.

3. Конструкторы, их виды, различия

- **По умолчанию** — Subject(string n = "") (с параметром по умолчанию), Action(...) — с обязательными параметрами.

- **С параметрами** — Student(string n, string s), Game(const Settings& s), Subject(string n).
- **Копирования** — неявный (компилятор), используется при push_back в вектор.
- **Различия:** конструктор по умолчанию позволяет создавать объект без аргументов; с параметрами — инициализирует поля сразу; конструктор копирования создаёт объект на основе другого. В коде явно определены только конструкторы с параметрами (и с параметрами по умолчанию), делегирования и перемещения нет.

4. Отладка программ

- Компиляция с предупреждениями, проверка синтаксиса.
- Пошаговое выполнение (breakpoints) в отладчике Visual Studio.
- Проверка ввода: cin.fail(), clearInput(), ограничение диапазонов (max/min), защита от пустых списков (chooseFrom).
- Контрольные точки: вывод showStatus(), промежуточные cout при сессии.
- Тестирование граничных случаев: 0/отрицательное число предметов, отмена выбора, неверный ввод, пустые файлы.
- Сравнение ожидаемого и фактического вывода, проверка файлов settings.txt и др.

5. Экстремальное программирование (XP)

- **Короткие итерации**, частая обратная связь.
- **Парное программирование** — два разработчика за одним кодом.
- **Тестирование в первую очередь** (TDD): сначала тесты, потом код.
- **Непрерывная интеграция** — частые сборки и прогон тестов.
- **Рефакторинг** — постоянное улучшение кода без изменения поведения.
- **Простота дизайна** — минимально необходимое решение.
- **Коллективное владение кодом**, стандарты кодирования, метафора системы.
- **Заказчик в команде** — быстрая постановка и уточнение требований.
- В лабораторной частично отражены: простота дизайна, постепенное наращивание функционала (настройки → игра → сессия), проверка ввода и итеративная доработка.